ACIT 2515 Reference Sheet

This is a compact reference. The railroad diagrams show common course patterns, not the full Python grammar.

Variables

Purpose

Use variables to give names to values so they can be read, updated, and passed between parts of a program.

Types

- Numeric: Used for arithmetic and measured quantities.
  - `int` : whole numbers, such as counts and port numbers.
  - `float` : decimal numbers, such as percentages and measurements.
  - `complex` : numbers with real and imaginary parts.
- Sequence: ordered values that can be iterated over and accessed by position.
  - `str` : text characters.
  - `list` : an ordered collection that can be changed.
  - `tuple` : an ordered collection that should remain fixed.
  - `range` : a generated sequence of integers, often used for repetition.
- Boolean: used for true/false conditions
  - `bool` is `True` or `False` .
- Set: a changeable collection of unique values, used for membership tests.
  - `set` : stores distinct values without positional indexes.
- Mapping: associates keys with values for lookup.
  - `dict` : stores key-value pairs, such as a hostname mapped to its status.
- None type: `None` represents the absence of a value.

Examples

```
port = 443                                                      # int
usage_percent = 82.5                                            # float
complex_value = 2 + 3j                                          # complex
hostname = "server01"                                           # str
ports = [22, 80, 443]                                           # list
coordinates = (49.2827, -123.1207)                              # tuple
retry_numbers = range(1, 4)                                     # range: 1, 2, 3
is_online = True                                                # bool
unique_ports = {22, 80, 443}                                    # set
service_status = {"hostname": "server01", "online": True}       # dict
owner = None                                                    # NoneType
```

Collections

Collections group values so they can be stored and processed together. A variable can refer to a collection just as it can refer to a single number.

Common Features

- Use a `for` loop to iterate over a collection.
- Use `len(collection)` to count its items.
- Use `value in collection` or `value not in collection` to test membership. For dictionaries, these tests check keys.
- Lists, sets, and dictionaries are mutable; strings, tuples, and ranges are immutable.

Sequences

Common Sequence Features

- Tuples, lists, strings, and ranges are distinct sequence types.
- Use a `for` loop to iterate over items in order.
- Use `len(sequence)` to count items.
- Use `value in sequence` or `value not in sequence` to test membership. Strings test for substrings; the other types test for items.
- Indexes start at zero; negative indexes count from the end.
- Slices use `[start:stop:step]`; the stop position is excluded.
- Tuples, lists, and strings support `+` to concatenate two sequences of the same type and `*` to repeat a sequence by an integer. Ranges do not support concatenation or repetition.
- Unpack items into names, such as `hostname, site = location`; the number of items must match the number of names.

Tuples

Purpose

Use a tuple to group related values that should be treated as a fixed sequence.

Notes

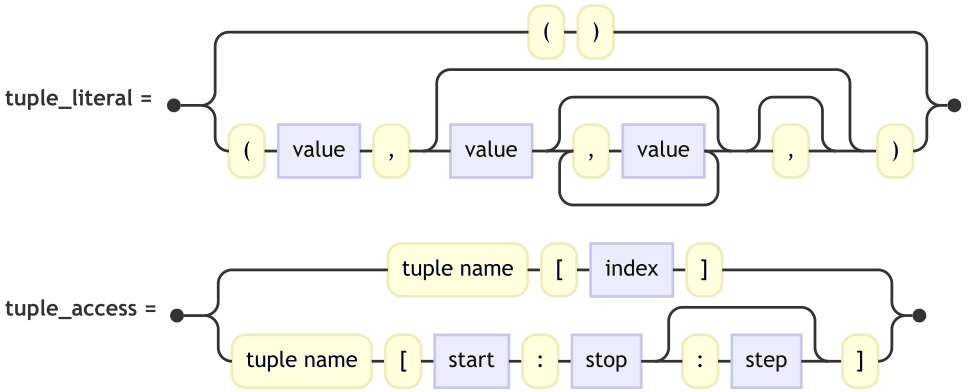
- Tuples are ordered and immutable; their items cannot be added, removed, or replaced.
- Tuples support the operations described under Common Sequence Features.

Methods Shared with Lists

- `.count(value)` returns the number of occurrences of a value.
- `.index(value[, start[, stop]])` returns the first matching index and raises `ValueError` when the value is absent. Square brackets here indicate optional arguments, not Python code. Pass the optional bounds by position, not by keyword; for example, `ports.index(443, 1)` searches from index `1`.

Syntax

Diagram



Examples

```
# Demonstrates tuple_literal and tuple_access; (443,) needs its trailing comma.
location = ("server01", "Vancouver")  # Tuple of two strings
single_port = (443,)                  # One-item tuple
server_name = location[0]             # "server01"
host_name, site_name = location       # "server01", "Vancouver"

ports = (22, 443, 443)
first_port = ports[0]                 # 22
last_port = ports[-1]                 # 443
first_two = ports[:2]                 # (22, 443)
port_count = len(ports)               # 3
has_https = 443 in ports              # True
https_count = ports.count(443)        # 2
https_index = ports.index(443)        # 1
combined = ports + (8080,)            # (22, 443, 443, 8080)
repeated = (443,) * 2                 # (443, 443)
```

Lists

Purpose

Use a list for an ordered group of values that may change and may contain duplicates.

Notes

- Lists are ordered and mutable; their items can be added, removed, or replaced.
- Lists support the operations described under Common Sequence Features and both methods described under Tuples: `.count()` and `.index()`.

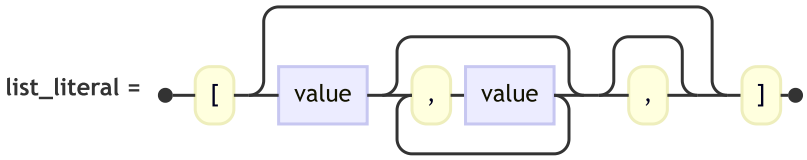
Additional List Methods

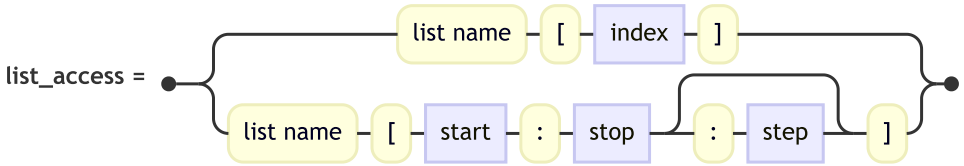
- `.append(value)` adds one object to the end and returns `None`.
- `.clear()` removes all items and returns `None`.
- `.copy()` returns a shallow copy of the list.
- `.extend(iterable)` appends every item from another iterable and returns `None`.
- `.insert(index, value)` inserts before the requested index and returns `None`.
- `.pop(index=-1)` removes and returns an item; it raises `IndexError` for an empty list or invalid index.

- `.remove(value)` removes the first matching item and raises `ValueError` when the value is absent.
- `.reverse()` reverses the list in place and returns `None`.
- `.sort(key=None, reverse=False)` sorts the list in place, returns `None`, and is stable. A `key` function controls comparison values; `reverse=True` sorts in descending order.

Syntax

Diagram





Examples

```
services = ["DNS", "SSH", "HTTPS"]       # Initial list
first_service = services[0]              # "DNS"
last_service = services[-1]              # "HTTPS"
first_two = services[0:2]                # ["DNS", "SSH"]
services.append("NTP")                   # Adds NTP at the end
services[1] = "SFTP"                     # Updates the second item
services.insert(1, "NTP")                # Inserts NTP at index 1
removed_service = services.pop()         # Removes and returns the last item

service_count = services.count("NTP")    # Number of NTP entries: 1
service_index = services.index("HTTPS")  # First index of HTTPS: 3
services.extend(["HTTP", "FTP"])         # Adds items from another list
service_copy = services.copy()           # Shallow copy
services.remove("FTP")                   # Removes the first FTP entry
services.reverse()                       # Reverses the list in place
services.sort()                          # Sorts the list in place
services.clear()                         # Removes all items; []
```

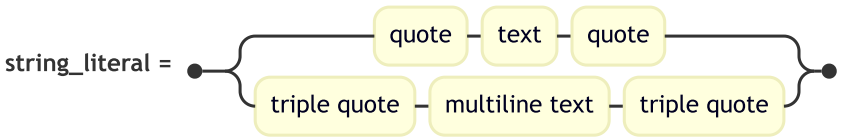
Strings

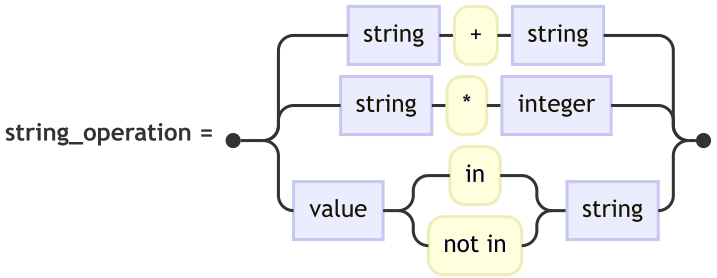
Purpose

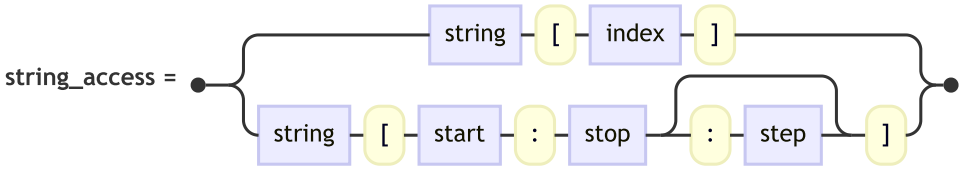
Use strings to store immutable text and process character sequences.

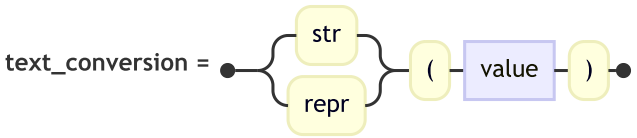
Syntax

Diagram









Examples

```
# Strings are immutable sequences; operations return new values.
single_line = "server01"
escaped = "line one\nline two"       # Contains a newline.

label = "host: " + single_line       # "host: server01"
separator = "-" * 8                  # "--------"
has_server = "server" in single_line  # True
has_router = "router" not in single_line  # True
length = len(single_line)            # 8
first = single_line[0]               # "s"
last = single_line[-1]               # "1"
prefix = single_line[:4]             # "serv"
suffix = single_line[4:]             # "er01"
every_second = single_line[::2]      # "svr0"
as_text = str(443)                   # "443"
debug_text = repr("server01")        # 'server01'
```

Notes

- Strings are immutable sequences of characters, not specialized lists.
- Strings support the operations described under Common Sequence Features.
- `in` and `not in` test for substring membership.
- `len(text)` returns the number of characters as an `int`.

- `str(value)` returns user-facing text.
- `repr(value)` returns a developer-oriented string representation.
- Escape sequences such as `\n` and `\t` have special meanings.

Common Methods

String methods return results without changing the original string.

- `.capitalize()` returns a copy with the first character uppercase and the remaining characters lowercase.
- `.lower()` and `.upper()` return copies converted to lowercase or uppercase.
- `.title()` returns a copy with the first character of each word uppercase and the remaining characters lowercase.
- `.strip()` removes leading and trailing whitespace; `.lstrip()` and `.rstrip()` remove it from only the left or right side.
- `.split(separator)` divides a string into a list of substrings. With no separator, it splits on runs of whitespace.
- `.join(iterable)` joins strings using the string as a separator, for example `", ".join(["DNS", "SSH"])`.
- `.replace(old, new)` returns a copy with occurrences of `old` replaced by `new`.
- `.find(substring)` returns the first matching index, or `-1` if not found.
- `.index(substring)` returns the first matching index, but raises `ValueError` if not found.
- `.count(substring)` returns the number of non-overlapping occurrences. Unlike tuple and list methods, string `.count()` and `.index()` search for substrings rather than individual items.
- `.startswith(prefix)` and `.endswith(suffix)` test how a string begins or ends.
- `.isalpha()`, `.isdigit()`, `.isalnum()`, and `.isspace()` test whether all characters are letters, digits, letters or digits, or whitespace, respectively. All four return `False` for an empty string.
- `.removeprefix(prefix)` and `.removesuffix(suffix)` return a copy with the specified prefix or suffix removed when present.

Triple-Quoted Strings

Use triple quotes (`'''` or `"""`) to write a string across multiple lines. Line breaks inside the quotes are included in the string.

```
message = """Service status:
DNS is online
SSH is offline"""
```

Raw Strings

Prefix a string literal with `r` to treat backslashes as ordinary characters, which is useful for Windows paths and regular expressions.

```
log_path = r"C:\logs\startup.log"
```

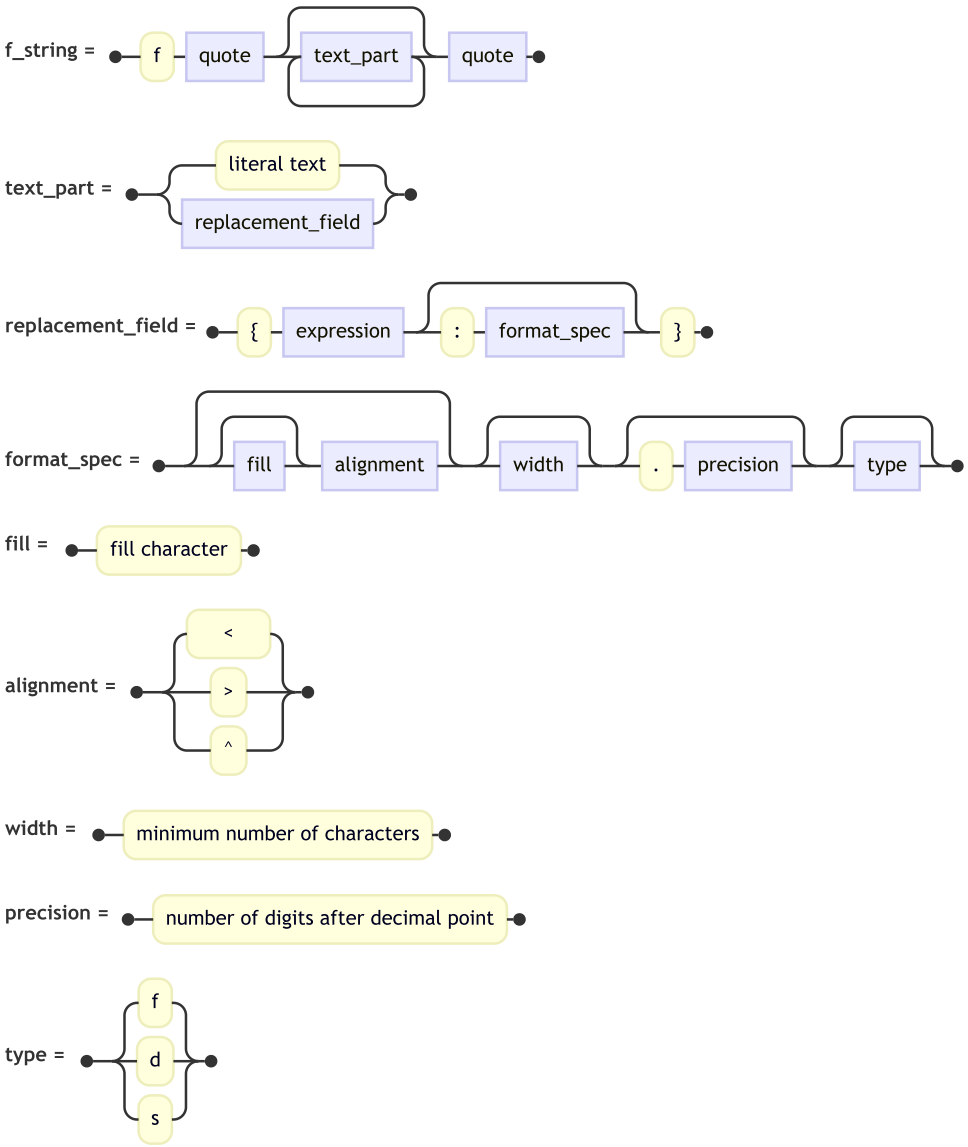
- Raw strings still follow normal quoting rules and cannot end with an odd number of backslashes.

F-Strings

Purpose

Use f-strings to build readable text that includes variable values and formatted numbers.

Syntax



Examples

```
# An f-string starts with f; braces insert values; : begins an optional format.
hostname = "server01"
disk_usage = 82.567

print(f"Checking {hostname}")       # Insert a value
print(f"Usage: {disk_usage:.2f}%")   # Two digits after the decimal: 82.57%
print(f"{disk_usage:08.2f}")        # Width 8, zero-filled: 0082.57
print(f"|{hostname:<12}|")          # Width 12, left aligned
print(f"|{hostname:>12}|")          # Width 12, right aligned
print(f"|{hostname:^12}|")          # Width 12, centered
# Output includes Checking server01, Usage: 82.57%, and aligned fields.
```

Notes

- A format width is a minimum field width in characters; longer values are not truncated.
- Alignment options are `<` for left, `>` for right, and `^` for centered.
- For floating-point values, `.2f` displays two digits after the decimal point. The `f` presentation type selects fixed-point formatting.

Ranges

Purpose

Use `range` to represent a sequence of integers for repetition, counting, and index-based iteration without writing out each integer.

Notes

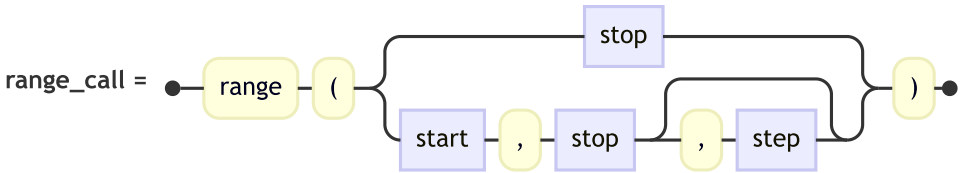
- A `range` is an immutable sequence of integers.
- Common forms are `range(stop)`, `range(start, stop)`, and `range(start, stop, step)`.
- `start` defaults to `0` and `step` defaults to `1`; `stop` is excluded.
- A negative step counts down; a zero step raises `ValueError`.
- Ranges support iteration, `len()`, indexing, slicing, and membership testing, as described under Common Sequence Features.
- Ranges do not support concatenation with `+` or repetition with `*`.
- A range stores its start, stop, and step rather than every integer.

Common Range Methods

- `.count(value)` returns the number of occurrences of a value: `0` or `1`.
- `.index(value)` returns the index of a value and raises `ValueError` when the value is absent.

Syntax

Diagram



Examples

```
attempts = range(1, 4)                # Values: 1, 2, 3
first_attempt = attempts[0]           # 1
last_attempt = attempts[-1]           # 3
first_two = attempts[:2]              # range(1, 3): values 1, 2
attempt_count = len(attempts)         # 3
has_second = 2 in attempts            # True
second_count = attempts.count(2)      # 1
second_index = attempts.index(2)      # 1
countdown = range(3, 0, -1)           # Values: 3, 2, 1

for attempt in attempts:
  print(attempt)  # 1, 2, 3
```

Sets

Purpose

Use a set to keep unique values or test membership without relying on position.

Notes

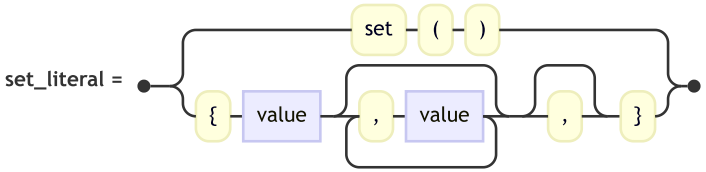
- Sets are mutable collections of unique, hashable values.
- Sets have no positional indexes and do not support indexing or slicing.
- Use `set()` to create an empty set; `{}` creates an empty dictionary.
- Use a `for` loop to iterate over values, `len(items)` to count them, and `value in items` to test membership.

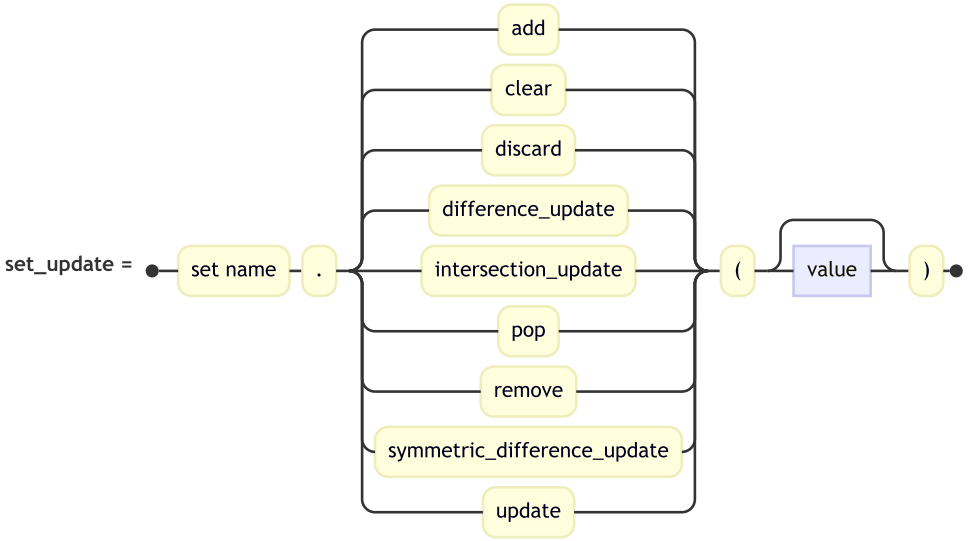
Common Set Methods

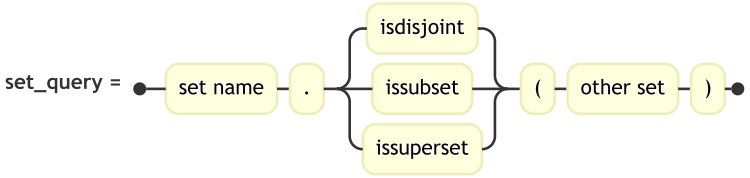
- `.add(value)`: adds one value to the set.
- `.clear()`: removes every value from the set.
- `.copy()`: returns a shallow copy of the set.
- `.difference(other)`: returns values in this set but not in `other`.
- `.difference_update(other)`: removes values found in `other` in place.
- `.discard(value)`: removes a value if present; otherwise does nothing.
- `.intersection(other)`: returns values present in both sets.
- `.intersection_update(other)`: keeps only shared values in place.
- `.isdisjoint(other)`: returns `True` when the sets share no values.
- `.issubset(other)`: returns `True` when every value is in `other`.
- `.issuperset(other)`: returns `True` when this set contains every value in `other`.
- `.pop()`: removes and returns an arbitrary value; it raises `KeyError` for an empty set.
- `.remove(value)`: removes a value and raises `KeyError` if it is absent.
- `.symmetric_difference(other)`: returns values in exactly one of the sets.
- `.symmetric_difference_update(other)`: keeps only values in exactly one set, modifying this set in place.
- `.union(other)`: returns a new set containing values from both sets.
- `.update(iterable)`: adds values from an iterable to the set.

Syntax

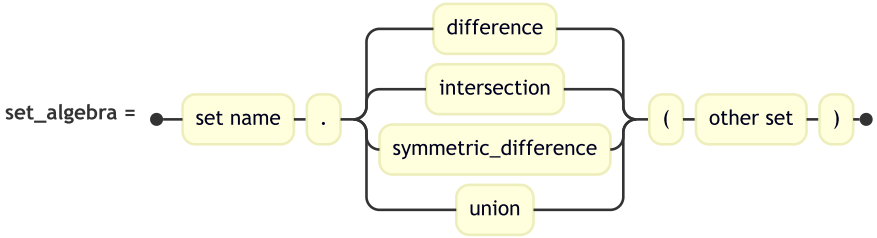
Diagram











Examples

```
# Demonstrates set_literal and set_update; duplicate 443 is stored once.
unique_ports = {22, 443, 443}
unique_ports.add(80)              # {22, 80, 443}
has_https = 443 in unique_ports  # True
empty_set = set()                # set()

web_ports = {80, 443}
secure_ports = {443, 8443}
shared_ports = web_ports.intersection(secure_ports)
# shared_ports is {443}.
```

Dictionaries

Purpose

Use a dictionary to associate values with unique, meaningful keys for lookup.

Notes

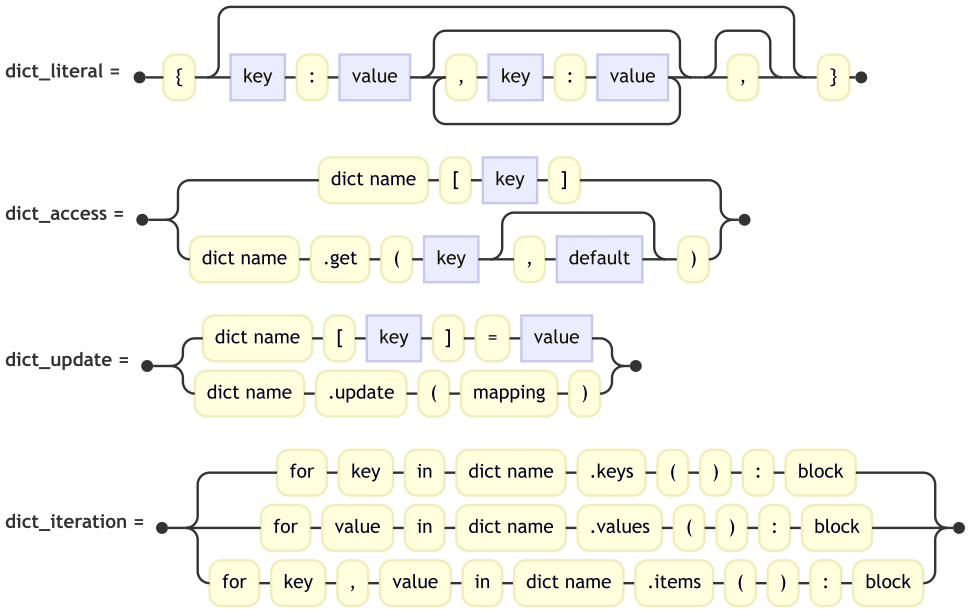
- Dictionary keys must be hashable; values can be duplicated.
- Access entries by key, not numeric position.
- Iterating over a dictionary directly is equivalent to iterating over its keys.
- `.keys()` supplies keys, `.values()` supplies values, and `.items()` supplies `(key, value)` pairs for unpacking in a `for` loop.
- Use `len(mapping)` to count entries.

Common Dictionary Methods

- `.clear()` : removes all key-value pairs and returns `None` .
- `.copy()` : returns a shallow copy of the dictionary.
- `.fromkeys(iterable, value=None)` : creates a new dictionary using the iterable's values as keys and the same default value for each key.
- `.get(key, default=None)` : returns a value or the default without raising `KeyError` when the key is absent.
- `.items()` : returns a view of `(key, value)` pairs.
- `.keys()` : returns a view of the dictionary's keys.
- `.pop(key)` : removes and returns the value for a key; it raises `KeyError` if the key is absent unless a default is supplied.
- `.popitem()` : removes and returns the most recently inserted `(key, value)` pair; it raises `KeyError` when the dictionary is empty.
- `.setdefault(key, default=None)` : returns the value for a key, inserting the default if the key is absent.
- `.update(mapping_or_iterable)` : adds or replaces key-value pairs in place and returns `None` .
- `.values()` : returns a view of the dictionary's values.

Syntax

Diagram



Examples

```
host = {"hostname": "server01", "ip": "192.168.1.10"}
host["port"] = 443                    # Adds the port field.
hostname = host["hostname"]           # "server01"
owner = host.get("owner", "unassigned")  # "unassigned"
host["status"] = "online"             # Adds the status field.

for field_name, field_value in host.items():
  print(f"{field_name}: {field_value}")  # Prints each field and value.

for field_name in host.keys():
  print(field_name)  # Prints each key.

for field_value in host.values():
  print(field_value)  # Prints each value.
```

Operators

Arithmetic Operators

Purpose

Use arithmetic operators to calculate numeric values and measurements.

Notes

Unary

- `+x` : leaves a number's sign unchanged; for example, `+(-5)` is `-5` .
- `-x` : negates a numeric value.

Binary

- `x + y` : adds two numbers.
- `x - y` : subtracts the right number from the left number.
- `x * y` : multiplies two numbers.
- `x / y` : divides and returns a floating-point result; division by zero raises `ZeroDivisionError` .
- `x // y` : performs floor division, rounding down; division by zero raises `ZeroDivisionError` .
- `x % y` : returns the remainder after division; a zero divisor raises `ZeroDivisionError` .
- `x ** y` : raises `x` to the power of `y` .

Examples

```
positive = +5                 # 5
negative = -5                 # -5
sum_value = 3 + 2             # 5
difference = 7 - 4            # 3
product = 3 * 4                # 12
quotient = 7 / 2               # 3.5
floor_quotient = 7 // 2        # 3
remainder = 7 % 2              # 1
power = 2 ** 3                 # 8
```

Comparison Operators

Purpose

Compare values and produce a Boolean result.

Notes

- `x == y` : tests whether two values are equal.
- `x != y` : tests whether two values are not equal.
- `x < y` : tests whether `x` is less than `y` .
- `x <= y` : tests whether `x` is less than or equal to `y` .
- `x > y` : tests whether `x` is greater than `y` .
- `x >= y` : tests whether `x` is greater than or equal to `y` .
- Comparisons can be chained, as in `1 <= port <= 1024` .

Examples

```
port = 443
port == 443                   # True
port != 80                    # True
port < 1024                   # True
port <= 443                   # True
port > 80                     # True
port >= 443                   # True
1 <= port <= 1024             # True
```

Boolean Operators

Purpose

Combine truth values to control decisions and repeated actions.

Notes

- `and` returns the first falsy operand, or the last operand if all are truthy.
- `or` returns the first truthy operand, or the last operand if all are falsy.
- `and` and `or` short-circuit: they stop evaluating operands once the result is known. Their results are not necessarily Boolean values.
- `not` reverses truth and always returns `True` or `False` .

Examples

```
port = 443
is_online = True                          # True
is_secure_port = True                     # True
should_check = is_online and is_secure_port   # should_check is True
unavailable = not is_online or port != 443    # unavailable is False

hostname = "" or "server01"               # "server01"
result = 0 and 10                         # 0; 10 is not evaluated
```

Membership Operators

Purpose

Test whether a value occurs in a sequence or collection; see Collections.

Notes

- For a dictionary, membership tests check keys.
- `in` and `not in` produce Boolean results.

Examples

```
open_ports = [22, 443]
needs_https_check = 443 in open_ports       # True
port_is_available = 8080 not in open_ports  # True

hostname = "server01.example.com"
has_domain = "example" in hostname           # True

service_status = {"DNS": "online", "SSH": "offline"}
has_dns_status = "DNS" in service_status     # True; checks dictionary keys
```

Identity Operators

Purpose

Test whether two names refer to the same object.

Notes

- Use `is` and `is not` for object identity, not value equality.
- Use `value is None` to test for `None` ; use `==` for value equality.

Examples

```
x = [ 1, 2, 3]
y = [ 1, 2, 3]
z = x
x == y                          # True
x is y                          # False
x == z                          # True
x is z                          # True
x.append(4)
x                               # [1,2,3,4]
y                               # [1,2,3]
z                               # [1,2,3,4]
owner = None                    # None
owner is None                   # True
```

Sequence Operators

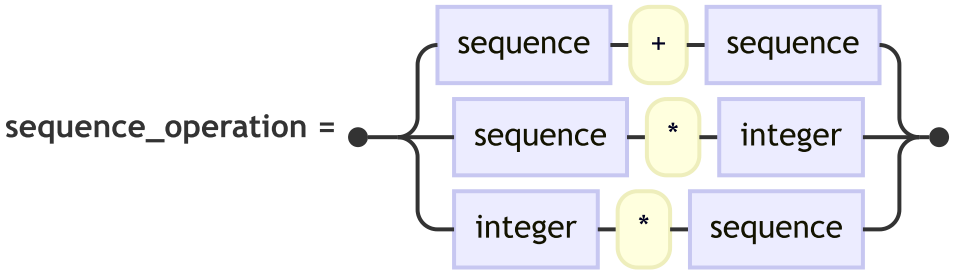
Purpose

Concatenate or repeat strings, lists, and tuples.

Notes

- `+` concatenates two sequences.
- `*` repeats a sequence by an integer.
- These operations create a new sequence.

Syntax



Examples

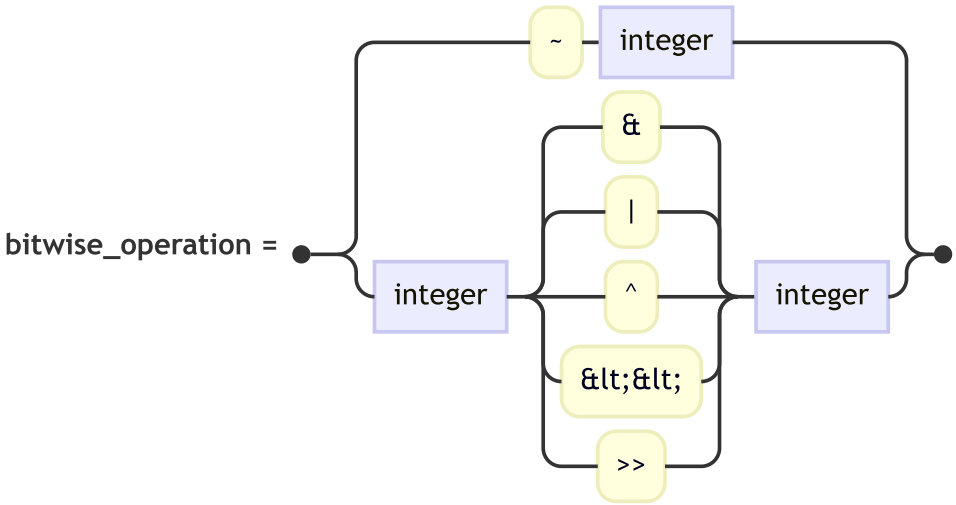
```
hostname = "server01"
label = "host: " + hostname       # "host: server01"
separator = "-" * 8                # "--------"
services = ["DNS"] + ["SSH"]       # ["DNS", "SSH"]
```

Bitwise Operators

Purpose

Operate on the individual bits of integer values. These operators are useful for working with flags and masks.

Syntax



Notes

- `&` sets a result bit when both corresponding bits are `1`.
- `|` sets a result bit when either corresponding bit is `1`.
- `^` sets a result bit when the corresponding bits differ.
- `~` inverts the bits of an integer. In Python, `~x` equals `-x - 1`; for example, `~5` is `-6`.
- `<<` shifts bits left; `>>` shifts bits right.
- These operators work with integers, not the Boolean operators `and`, `or`, and `not`.

Examples

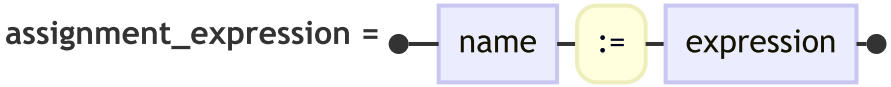
```
permissions = 0b0101
read_flag = 0b0001
permissions & read_flag  # 0b0101 & 0b0001 = 1 (0b0001): read flag is set
permissions | 0b0010     # 0b0101 | 0b0010 = 7 (0b0111): set write flag
permissions ^ 0b0100     # 0b0101 ^ 0b0100 = 1 (0b0001): toggle execute flag
1 << 3                   # 0b0001 << 0b0011 = 8 (0b1000)
8 >> 2                   # 0b1000 >> 0b0010 = 2 (0b0010)
~5                       # ~0b0101 = -6 (Python: bin(-6) is -0b110)
```

Walrus Operator

Purpose

Assign a value to a variable as part of a larger expression.

Syntax



Notes

- `:=` is called the assignment expression operator.

- The expression both assigns the value to the name and evaluates to that value.
- Use it when the assigned value is also needed in the surrounding expression; ordinary `=` assignment is clearer otherwise.

Examples

```
hostname = "server01"
if (name_length := len(hostname)) > 7:
  print(name_length)  # 8
```

Augmented Assignment Operators

Purpose

Update a variable by applying an operation and assigning the result back to that variable.

Examples

```
value = 1
value += 2        # 3
value -= 1        # 2
value *= 3        # 6
value /= 4        # 1.5
value //= 1       # 1.0
value %= 0.75     # 0.25
value **= 2       # 0.0625

bits = 10
bits &= 6         # 2
bits |= 8         # 10
bits ^= 3         # 9
bits <<= 1        # 18
bits >>= 2        # 4
```

Notes

- Arithmetic augmented assignments are `+=`, `-=`, `*=`, `/=`, `//=`, `%=`, and `**=`.
- Bitwise augmented assignments are `&=`, `|=`, `^=`, `<<=`, and `>>=`.
- `@=` performs matrix multiplication and assignment for types that support matrix multiplication, for example `matrix @= transform`.
- For immutable values such as numbers, an augmented assignment calculates a new value and reassigns the variable.
- For mutable types, it may update the object in place.

Operator Precedence

Purpose

Determine which operators are evaluated first when parentheses are absent.

Order

The following order is highest precedence to lowest precedence:

1. Parenthesized expressions and displays: `( ... )`, `[ ... ]`, `{key: value}`, `{value, ...}`
2. Subscription, slicing, calls, and attribute references: `x[index]`, `x[start:stop]`, `x(arguments)`, `x.attribute`
3. Await expression: `await x`
4. Exponentiation: `**`
5. Unary positive, negative, and bitwise NOT: `+x`, `-x`, `~x`
6. Multiplication, division, floor division, and remainder: `*`, `/`, `//`, `%`
7. Addition and subtraction: `+`, `-`
8. Shifts: `<<`, `>>`
9. Bitwise AND: `&`
10. Bitwise XOR: `^`
11. Bitwise OR: `|`
12. Comparisons, membership, and identity: `<`, `<=`, `>`, `>=`, `!=`, `==`, `in`, `not in`, `is`, `is not`
13. Boolean `not`
14. Boolean `and`
15. Boolean `or`
16. Conditional expression: `value_if_true if condition else value_if_false`
17. Lambda expression: `lambda parameters: expression`
18. Assignment expression: `name := expression`

Notes

- Operators at the same level generally evaluate left to right, except exponentiation and conditional expressions, which group right to left.
- Comparisons, membership tests, and identity tests have the same precedence and can be chained.
- Parentheses make grouping explicit.

Truthy and Falsy Values

Purpose

Understand how Python treats values as true or false in conditions and Boolean expressions.

Notes

- Python considers an object truthy by default unless its type defines it as falsy. See the Python truth value testing documentation.
- The Boolean value `True` is truthy. The string `"True"` is also truthy because it is non-empty, not because of the word it contains.
- `None` and `False` are falsy.
- Non-zero numbers are truthy.
- Zero numeric values are falsy, examples include `0`, `0.0`, and `0j`.
- Non-empty sequences and collections are truthy; examples include `[1]`, `(1,)`, `{1}`, `{"key": "value"}`, `"server"`, and `range(1)`.
- Empty sequences and collections are falsy, examples include `[]`, `()`, `{}`, `set()`, `""`, and `range(0)`.
- Use `bool(value)` to inspect the truth value explicitly.

Examples

```
a = 0
if a:
  print(a)  # No output because 0 is falsy.

# Non-zero numbers are truthy, so the body runs.
a = 5
if a:
  print(a)  # 5

empty_services = []
zero_value = 0
no_owner = None

bool(empty_services)  # False
bool(zero_value)      # False
bool(no_owner)        # False
bool(object())        # True
bool(["DNS"])         # True
bool(5)               # True
```

Flow Control

Purpose

Flow control determines the order in which a program's statements run. Use it to choose between actions and to repeat actions.

Notes

- By default, Python runs statements one at a time from top to bottom (sequence).
- Selection chooses which block runs: `if`, `elif`, and `else`, or a conditional expression.
- Iteration repeats a block: `for` and `while` loops, with `break` and `continue` to change how a loop proceeds.
- Indentation defines each block, and a colon begins the indented block.
- Functions, recursion, and exceptions also change the flow of a program; see `Functions`, `Recursion`, and `Exceptions`.

Conditional Statement: `if`

Purpose

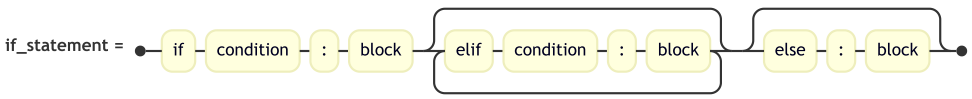
Use conditional statements to choose which block of code runs based on a condition.

Notes

- Indentation defines each block.
- A colon begins the indented block.

Syntax

Diagram



Examples

```
if usage_percent >= 90:
   # Statements in the if block must be indented to the same level
   status = "critical"
elif usage_percent >= 75: # Follows if_statement: elif may repeat
   status = "warning"
else:                     #the final else is optional.
   status = "normal"
# With usage_percent = 82.5, status is "warning".
```

Conditional Expressions: `if` Ternary Operator

Purpose

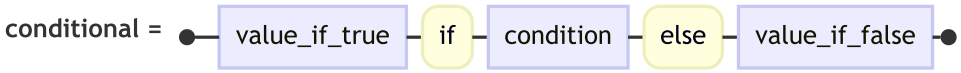
Choose one value from two alternatives based on a condition.

Notes

- Use a conditional expression for a simple choice.
- Use a regular `if` statement for multiple steps or branches.

Syntax

Diagram



Examples

```
status = "online" if is_online else "offline"  # "online"
```

for Loops

Purpose

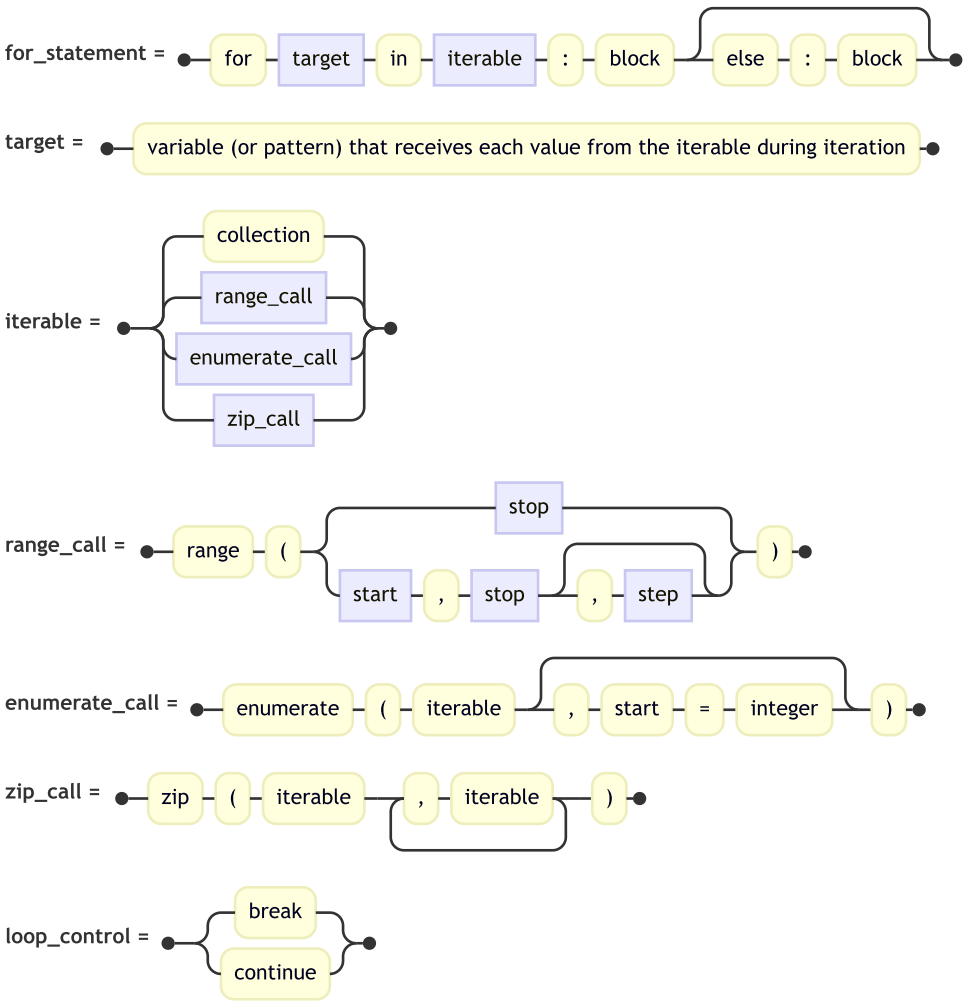
Use a `for` loop to process every item in a collection or sequence.

Notes

- The `else` block runs when the iterable is exhausted, including when it is empty. It does not run if `break`, `return`, or an exception exits the loop.
- `break` exits the nearest enclosing loop immediately; the loop does not process any remaining items.
- `continue` skips the rest of the current iteration and starts the next one.
- The stop value in `range()` is exclusive
- `enumerate(iterable, start=0)` returns an iterator of `(index, item)` pairs. The default index is `0`; use `start=1` for human-readable numbering.
- `zip(iterable1, iterable2, ...)` returns an iterator of tuples containing one item from each input iterable at the same position.
- `zip()` stops when the shortest input is exhausted; extra items in longer inputs are not included.
- Both `zip` and `enumerate` return iterators, and can be used in a `for` loop or converted to a list with `list()`.

Syntax

Diagram



Examples

```
for char in "test":
  print(char) #iterates over string printing one char per line

for attempt in range(1, 4):
  print(f"Connection attempt {attempt}")  # Attempts 1, 2, and 3

services = ["Spooler", "W32Time", "WinRM"]
for service_name in services:
  print(f"Checking {service_name}")  # Checking each service name

# enumerate() supplies an index and item during each iteration.
for position, service_name in enumerate(services, start=1):
  print(f"{position}. {service_name}")  # 1. Spooler, 2. W32Time, 3. WinRM

# zip() pairs items from multiple iterables in order.
statuses = ["running", "stopped", "running"]
for service_name, service_status in zip(services, statuses):
  print(f"{service_name}: {service_status}")

# Outputs
# Spooler: running
# W32time: stopped
# WinRM: running

for service_name in ["DNS", "SSH", "NTP", "HTTPS", "SMTP"]:
  if service_name == "SSH":
    continue # skips the rest of the current for loop.
  if service_name == "HTTPS":
    break # stops the nearest loop immediately.
  print(service_name)

# Outputs:
# DNS
# NTP
```

while Loops

Purpose

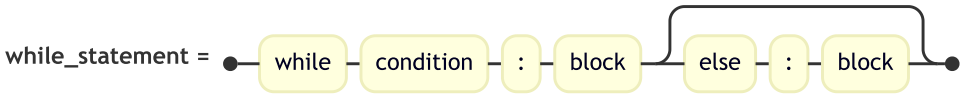
Use a `while` loop to repeat a block while a condition remains true, especially when the number of repetitions is not known in advance.

Notes

- Update a value used by the condition so it can eventually become false; otherwise, the loop may never stop.

Syntax

Diagram



Examples

```
attempt = 1
while attempt <= 3:
  print(attempt)  # 1, then 2, then 3
  # incrementing attempt lets the condition become false, otherwise it will iterate indenitely
  attempt += 1
else:
  print("Out of Attempts") #executes after the while loop exists normally
```

Functions

Purpose

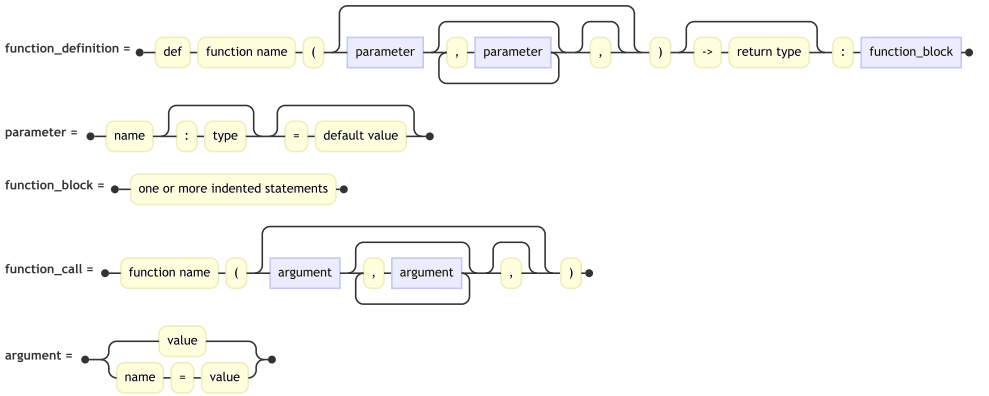
Use functions to package reusable behavior behind named inputs and return values.

Notes

- Parameters are names in a function definition; arguments are values in a function call.
- `return` sends a result back. Without `return`, a function returns `None`.
- A function that receives a list can mutate the shared list; those changes are visible to the caller.
- Reassigning the parameter name only changes the local name. Use `items.copy()` before changing a private copy.

Syntax

Diagram



Examples

```python
def free_space(total_gb: float, used_gb: float) -> float:
  """Return the available disk space in gigabytes."""
  return total_gb - used_gb


available = free_space(total_gb=500.0, used_gb=125.5)  # 374.5

def host_label(hostname: str, site: str | None = None) -> str:
  """Return a hostname label, optionally including its site."""
  if site is None:
    return hostname  # Example: "server01"
  return f"{hostname} ({site})"  # Example: "server01 (Vancouver)"

# Passing a list shares the mutable object, so appending affects the caller's list.
def add_service(services: list[str], service_name: str) -> None:
  """Add a service name to the caller's list."""
  services.append(service_name)

# Alterantively make a copy, so the caller's list is unaffected
def sorted_service_copy(services: list[str]) -> list[str]:
  """Return a sorted copy without changing the caller's list."""
  result = services.copy()
  result.sort()
  return result

# The copy is sorted separately, leaving service_names in its original order.
service_names = ["WinRM", "DNS"]
add_service(service_names, "SSH")
sorted_names = sorted_service_copy(service_names)  # ["DNS", "SSH", "WinRM"]
# service_names remains ["WinRM", "DNS", "SSH"].
```

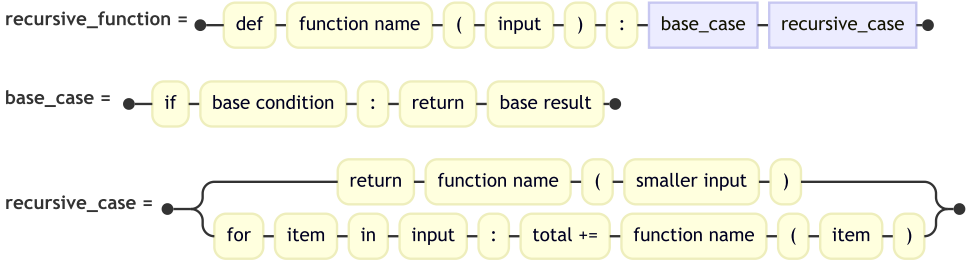
Recursion

Purpose

Use recursion to solve a problem by applying the same function to smaller or simpler parts of the input.

Syntax

Diagram



Examples

```python
def sum_nested(data: int | list) -> int:
  """Return the sum of integers in a possibly nested list."""
  # Base case: integers return directly; the loop recursively sums child items.
  if isinstance(data, int):  # Base case
    return data

  total = 0
  for item in data:  # Recursive case: process each child
    total += sum_nested(item)
  return total  # Example: sum_nested([1, [2, 3]]) returns 6.
```

Notes

- Every recursive function needs a base case that stops and a recursive case that moves toward it.
- Each call has its own local variables.
- Missing a base case, or failing to make progress toward it, can cause `RecursionError`.

Higher Order Functions

Purpose

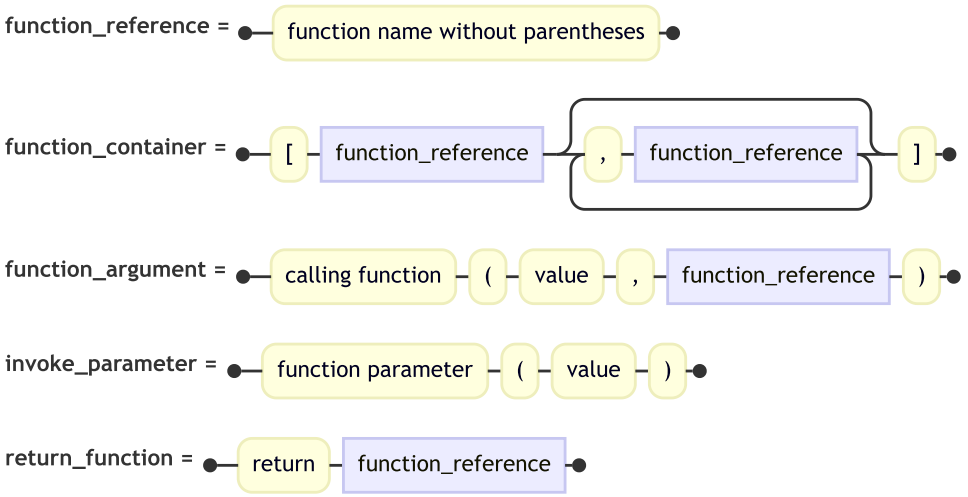
Use a function that accepts another function as an argument or returns a function. This lets programs reuse code while changing the behavior it applies.

Notes

- Functions are values: they can be assigned to names, stored in collections, passed as arguments, and returned.
- `check` refers to a function; `check()` calls it.
- A nested function is defined inside another function.
- A closure lets a nested function keep access to variables from its enclosing function, even after that enclosing function returns.
- Use `nonlocal` only when reassigning an enclosing function's variable; simply reading it does not require `nonlocal`.

Syntax

Diagram



Functions as Arguments

Pass a function without parentheses so the receiving function can call it.

```
def normalize_hostname(text: str) -> str:
  """Return a hostname in lowercase without surrounding whitespace."""
  return text.strip().lower()


def host_label(hostname, transform):
  """Build a label using the supplied text transformation."""
  return f"Host: {transform(hostname)}"


print(host_label(" SERVER01 ", normalize_hostname))  # Host: server01
```

Functions That Return Functions

Return the nested function without calling it. This example's closure remembers the configured limit.

```
def make_usage_check(limit: float):
  """Return a function that checks usage against the supplied limit."""
  def is_high(usage: float) -> bool:
    """Return whether usage meets or exceeds the saved limit."""
    return usage >= limit

  return is_high


warning_check = make_usage_check(80)
print(warning_check(85))  # True
print(warning_check(50))  # False
```

Decorators

A function decorator accepts a function and returns a replacement, usually a wrapper that adds behavior before or after calling the original function.

- `@log_call` above a function definition has the same effect as defining the function and then assigning `check_host = log_call(check_host)`.
- The decorator is applied when the function is defined; the wrapper runs each time the decorated function is called.
- `*args` and `**kwargs` collect and forward positional and keyword arguments.
- Return the original function's result so callers can still use it.
- `@wraps(func)` preserves metadata such as the original name and docstring.

```
from functools import wraps


def log_call(func):
  """Wrap a function to announce each call."""
  @wraps(func)
  def wrapper(*args, **kwargs):
    """Log the call and return the original function's result."""
    print(f"Calling {func.__name__}")
    return func(*args, **kwargs)

  return wrapper


@log_call
def check_host(hostname: str) -> str:
  """Return a host check message."""
  return f"Checking {hostname}"


print(check_host("server01"))
# Calling check_host
# Checking server01
```

Classes and Objects

Purpose

Use classes to group related data and behavior.

Notes

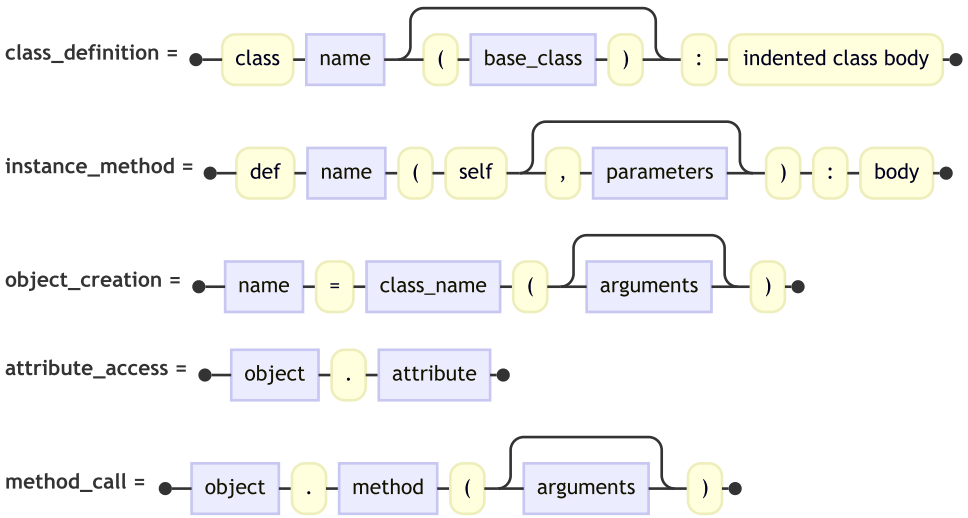
- A class defines a type; an object (instance) is a particular example of that type.
- Calling a class, such as `Service("HTTPS", 443)`, creates an instance.
- Attributes store state
- Methods define behavior.
- `__init__()` initializes an instance after it is created and returns `None`.
- `self` refers to the current instance. Python implicitly supplies it when calling an instance method
- Put per-instance mutable values, such as lists, in `__init__()`.
- Assign `self.attribute` inside a method to store an instance attribute.
- A class attribute is defined in the class body and shared through the class.
- Two names can refer to the same object. `is` tests identity; `==` tests equality, which a class can customize.

Common Method Types

- Instance methods receive `self` and work with an object's state.
- `@classmethod` methods receive the class as `cls`. Use them for alternative constructors or class-level state
  - `return cls(...)` creates an instance.
- `@staticmethod` methods receive neither `self` nor `cls` automatically.
  - Use them for a related utility that needs no instance or class state.

Syntax

Diagram



Examples

```python
class Service:
  """Represent a named network service."""

  category = "network"

  def __init__(self, name: str, port: int) -> None:
    """Initialize the name and port."""
    self.name = name
    self.port = port

  def describe(self) -> str:
    """Return a short service description."""
    return f"{self.name} uses port {self.port}"

  @classmethod
  def from_text(cls, text: str):
    """Create a service from a name:port string."""
    name, port_text = text.split(":")
    return cls(name, int(port_text))

  @staticmethod
  def valid_port(port: int) -> bool:
    """Return whether a port is in the valid range."""
    return 1 <= port <= 65535

  def __str__(self) -> str:
    """Return a user-facing description."""
    return self.describe()

  def __repr__(self) -> str:
    """Return a developer-facing representation."""
    return f"Service({self.name!r}, {self.port!r})"


service = Service("HTTPS", 443)
other = Service.from_text("DNS:53")
print(service.name)                # HTTPS
print(other.describe())            # DNS uses port 53
print(Service.category)            # network
print(Service.valid_port(443))     # True
print(service)                     # HTTPS uses port 443
print(repr(service))               # Service('HTTPS', 443)
```

Special Methods

Python calls special (dunder) methods to implement familiar operations. Use the operation, such as `len(obj)`, rather than calling the method directly.

- `__str__(self)`: readable text for `str(obj)` and `print(obj)`.
- `__repr__(self)`: developer-oriented text for `repr(obj)` and object display inside collections. Ideally include the class name and important state.
- The following dunder methods customize the corresponding operations. Inherited methods also apply; a class does not need to define them all. For example, objects support identity-based equality by default, and `in` can use iteration when `__contains__` is not defined.
  - `__eq__(self, other)`: `==`.
  - `__lt__(self, other)`: `<`.
  - `__le__(self, other)`: `<=`.
  - `__gt__(self, other)`: `>`.
  - `__ge__(self, other)`: `>=`.
  - `__add__(self, other)`: `+`.
  - `__sub__(self, other)`: `-`.
  - `__mul__(self, other)`: `*`.
  - `__truediv__(self, other)`: `/`.
  - `__len__(self)`: `len(obj)`.
  - `__getitem__(self, key)`: `obj[key]`.
  - `__setitem__(self, key, value)`: `obj[key] = value`.
  - `__contains__(self, item)`: `item in obj`.

Encapsulation and Abstraction

Notes

- Encapsulation keeps state and the methods that manage it together.

- Abstraction exposes a simple interface while hiding implementation details.
- Access Conventions
  - `name` is public: outside code can use it as part of the class's interface.
  - `_name` signals internal use by convention, not enforced access protection. code outside of the class shouldn't use these and use public methods or properties instead
  - `__name` triggers name mangling, such as `_Service__name`. This helps avoid subclass naming conflicts: a subclass's own `__name` does not overwrite the parent's attribute.
- `@property` makes a getter accessible as `obj.port`, *without parentheses*.
- `@port.setter` handles assignment to `obj.port` and can validate changes. A property without a setter is read-only through that property.

Examples

```
class Listener:
  """Manage a listener's port through a validated property."""

  def __init__(self, port: int) -> None:
    """Initialize the port using the property's validation."""
    self.port = port

  @property
  def port(self) -> int:
    """Return the configured port."""
    return self._port

  @port.setter
  def port(self, value: int) -> None:
    """Set the port or raise ValueError if it is outside the valid range."""
    if not 1 <= value <= 65535:
      raise ValueError("port must be between 1 and 65535")
    self._port = value


listener = Listener(8080)
listener.port = 8443
print(listener.port)  # 8443
```

Object Relationships

Notes

- Composition is an owned "has-a" relationship: a containing object creates and manages its parts.
- Aggregation is an independent "has-a" relationship: a containing object holds references to objects created separately, which can be shared.
- These are design relationships, not special Python syntax. Removing a container does not destroy a part if another reference to that part remains.

Examples

This example uses the `Service` class defined above.

```
class ScanRecord:
  """Store a single scan message."""

  def __init__(self, message: str) -> None:
    """Initialize the message."""
    self.message = message


class ScanReport:
  """Own scan records and refer to independently created services."""

  def __init__(self, services: list[Service]) -> None:
    """Keep service references and initialize an independent record list."""
    self.services = services
    self.records: list[ScanRecord] = []

  def add_record(self, message: str) -> None:
    """Create and retain a record owned by this report."""
    self.records.append(ScanRecord(message))


service = Service("SSH", 22)
report = ScanReport([service])  # Aggregation: service exists independently.
report.add_record("Scan queued")  # Composition: report creates the record.
```

Inheritance, Polymorphism, and Abstract Base Classes

Notes

- Inheritance models "is-a": `class Child(Parent)` inherits behavior from its parent, reducing duplication.
- Override a method by defining the same name in a subclass.
  - Keep its calling interface compatible with the inherited method.
- If a subclass defines `__init__()`, call `super().__init__(...)` to perform required parent setup. Without its own initializer, it inherits one.
- Extend rather than replace behavior with `result = super().method(...)`, then add the subclass's work.
- Polymorphism lets the same method call use each object's implementation. Callers need a shared interface rather than separate type checks.
- `isinstance(obj, Parent)` also matches subclasses; `isinstance(obj, (TypeA, TypeB))` matches either type.
- In comparison methods, return `NotImplemented` for unsupported operand types so Python can try the other operand's implementation.
- Inherit from `ABC` and mark required methods with `@abstractmethod` to define an abstract interface.
- A class with unimplemented abstract methods cannot be instantiated; attempting to do so raises `TypeError`. Inheriting from `ABC` alone does not prevent instantiation.
- An abstract method may contain shared code; an override can call it with `super()`. Raising `NotImplementedError` alone does not enforce abstractness.
- For an abstract property, put `@property` above `@abstractmethod`; implement the property in a concrete subclass.
- Multiple inheritance lists several bases: `class Child(Mixin, Parent)`. A mixin adds focused, reusable behavior rather than representing a main type.
- The method resolution order (MRO) determines attribute lookup. Inspect it with `ClassName.__mro__`. Base order matters, but shared ancestors are not simply searched depth-first.
- `super()` follows the MRO after the class containing the method, not necessarily a single named parent.

Examples

```
from abc import ABC, abstractmethod


class StatusFormatter(ABC):
  """Define an interface for formatting service status."""

  def __init__(self, hostname: str) -> None:
    """Store the host shared by all formatters."""
    self.hostname = hostname

  @property
  @abstractmethod
  def separator(self) -> str:
    """Return the separator used by this format."""
    pass

  @abstractmethod
  def format_status(self, online: bool) -> str:
    """Return a shared host prefix for subclasses to extend."""
    return self.hostname + self.separator


class TextStatus(StatusFormatter):
  """Format status as a sentence."""

  def __init__(self, hostname: str, label: str) -> None:
    """Initialize the inherited host and a custom label."""
    super().__init__(hostname)
    self.label = label

  @property
  def separator(self) -> str:
    """Return the separator for sentence output."""
    return " is "

  def format_status(self, online: bool) -> str:
    """Return a labeled status sentence."""
    state = "online" if online else "offline"
    return f"{self.label}: {super().format_status(online)}{state}"


class CompactStatus(StatusFormatter):
  """Format status as a short record."""

  @property
  def separator(self) -> str:
    """Return the separator for compact output."""
    return ","

  def format_status(self, online: bool) -> str:
    """Return a host and numeric status."""
    return super().format_status(online) + str(int(online))


formatters = [
  TextStatus("gateway01", "Network"),
  CompactStatus("gateway01"),
]
for formatter in formatters:
  print(formatter.format_status(True))
# Network: gateway01 is online
# gateway01,1
print(isinstance(formatters[0], StatusFormatter))  # True
```

Multiple inheritance lets `LoggedStatus` combine the logging behavior supplied by `LogMixin` with the formatting behavior inherited from `CompactStatus`.

```
class LogMixin:
  """Add a simple console logging helper."""

  def log(self, message: str) -> None:
    """Display a log message."""
    print(f"Log: {message}")


class LoggedStatus(LogMixin, CompactStatus):
  """Log each request before formatting it."""

  def format_status(self, online: bool) -> str:
    """Log and delegate to the next formatter in the MRO."""
    self.log("Formatting status")
    return super().format_status(online)


print(LoggedStatus("gateway01").format_status(True))
# Log: Formatting status
# gateway01,1
# MRO: LoggedStatus, LogMixin, CompactStatus, StatusFormatter, ABC, object
```

Exceptions

Purpose

Use exceptions to report or handle runtime problems that prevent normal operation, by interrupting normal program flow.

Notes

Exception Flow and Tracebacks

- When an exception is raised, the remaining statements in that `try` block are skipped.
- Python checks the `except` clauses from top to bottom and runs only the first matching handler. A handler for a parent exception type also matches its subclasses, so put more specific handlers first.
- If no handler matches, the exception propagates to the calling code. If it remains unhandled, the program normally stops and displays a traceback.
- A traceback shows the calls and source lines involved; its final line gives the exception type and message. Start there, then trace back to the cause.
- After a handler finishes normally, execution continues after the whole `try` statement, not at the statement that failed.

Specific Handlers and Propagation

- Catch specific exceptions only where the program can recover, and keep `try` blocks small.
- Avoid bare `except:` and unnecessarily broad `except Exception:` handlers: they can hide unexpected problems. A bare handler also catches `KeyboardInterrupt` and `SystemExit`.
- Do not silently ignore an error with `pass`; give a useful message or let the exception propagate.

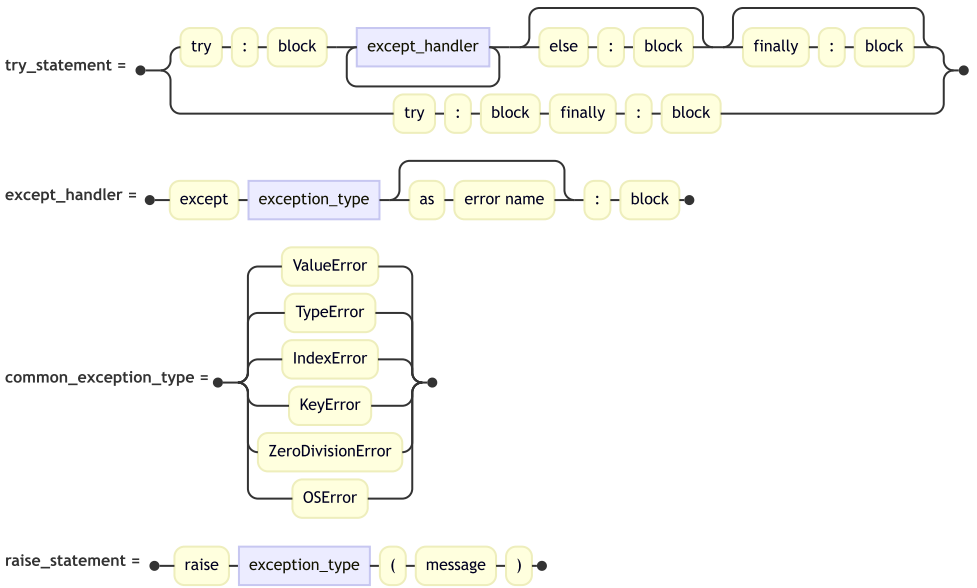
Common Exception Types

- `ValueError`: invalid value or conversion.
- `TypeError`: incompatible types.
- `IndexError`: invalid sequence position.
- `KeyError`: missing dictionary key.
- `ZeroDivisionError`: division or remainder with a zero divisor.
- `FileNotFoundError`: requested file is missing.
- `PermissionError`: operation is not permitted.
- `OSError`: operating-system or file operation failure.

`FileNotFoundError` and `PermissionError` are subclasses of `OSError`.

Syntax

Diagram



Examples

Handling Errors: `try` and `except`

- Put the operation that may fail inside `try`.
- Catch a specific expected exception with `except`.
- `except ValueError as error` binds the exception object to `error` inside that handler; printing it displays its message.
- Use multiple handlers when different problems need different responses.
- Use a tuple, such as `except (OSError, UnicodeError) as error:`, when different exception types should share the same handler.

```
try:
  with open("service_port.txt", encoding="utf-8") as config_file:
    port = int(config_file.read())
except FileNotFoundError:
  print("Create service_port.txt before checking the service.")
except PermissionError:
  print("The current account cannot read service_port.txt.")
except ValueError:
  print("The configuration must contain a whole-number port.")
except OSError as error:
  print(f"Could not read the configuration: {error}")
else:
  print(f"Configured port: {port}")
```

Continuing on Success: `else`

- `else` runs only when the `try` block finishes normally, without an exception or an early `return`, `break`, or `continue`.
- Put work that depends on success here rather than after the whole statement.
- Exceptions raised in `else` are not handled by the preceding `except` clauses.

```
port_text = "443"
try:
  port = int(port_text)
except ValueError as error:
  print(f"Port must be a whole number: {error}")
else:
  print(f"Checking port {port}")  # Checking port 443
```

Cleaning Up: `finally`

- `finally` runs when execution leaves the statement, whether the operation succeeds, a handler runs, or an exception remains unhandled. It also runs before a pending `return`, `break`, or `continue` takes effect.
- Use it to release resources or restore state. For files, prefer a `with` statement when possible because it closes the file automatically.
- Cleanup does not handle an exception: an unhandled exception continues propagating after `finally`.
- Avoid `return`, `break`, or `continue` in `finally`; they can suppress an exception or override an earlier return.

```
# Open before try: if opening fails, no file was acquired to close.
log_file = open("service.log", encoding="utf-8")
try:
  contents = log_file.read()
except (OSError, UnicodeError) as error:
  print(f"Could not read the log: {error}")
else:
  print(f"Read {len(contents)} characters.")
finally:
  log_file.close()  # Closes the file even if reading raises an exception.
```

Reporting Invalid Values: `raise`

- Use `raise` with an appropriate built-in exception and an informative message when a function cannot fulfill its contract.
- Let the caller decide how to respond rather than printing an error and returning a value that looks like success.

```
def validate_port(port: int) -> int:
  """Return a valid network port.

  Args:
    port: Port number to check.

  Returns:
    The validated port number.

  Raises:
    ValueError: If the port is outside the range 1 through 65535.
  """
  if not 1 <= port <= 65535:
    raise ValueError("port must be between 1 and 65535")
  return port


try:
  checked_port = validate_port(70000)
except ValueError as error:
  print(f"Invalid port: {error}")
else:
  print(f"Validated port {checked_port}")
# Invalid port: port must be between 1 and 65535
```

Code Documentation

Use comments, type annotations, and docstrings to make code easier to understand.

They serve different purposes: comments explain decisions, annotations describe expected types, and docstrings document how code is used.

Comments

Purpose

Use comments to explain intent or non-obvious decisions without changing what the program does.

Notes

- A comment starts with `#` outside a string and continues to the end of the line. Python ignores it during execution.
- Put comments on their own lines or after code, separated by at least two spaces.
- Explain why something is done rather than restating obvious code.
- Triple-quoted strings are strings, not block comments. They are docstrings only when placed as the first statement in a module, class, or function.

Syntax

```
# Comment on its own line
statement  # Comment after code
```

Examples

```
# Port 443 is required because this service accepts HTTPS connections.
service_port = 443
```

Type Annotations

Purpose

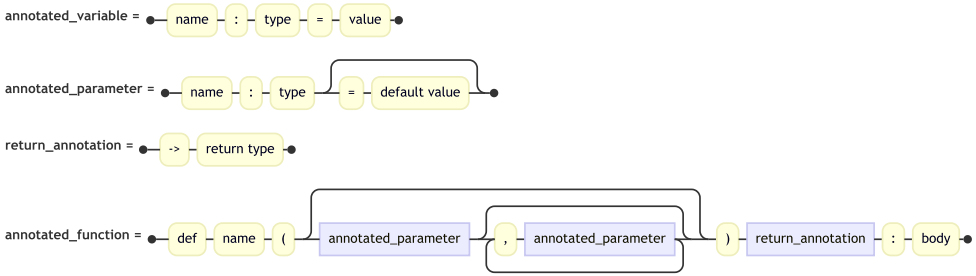
Use type annotations to describe the types of values expected by variables, function parameters, and return values.

Notes

- Parameter annotations describe expected argument values; the return annotation describes the function result.
- Common collection annotations include `list[str]`, `dict[str, int]`, `tuple[str, int]`, and `set[str]`.
- `str | None` means a string or `None`; a default such as `site = None` makes the argument optional for callers.
- `-> None` describes a function that does not return a useful value.
- Annotations guide readers and type checkers; Python does not automatically enforce them at runtime.

Syntax

Diagram



Examples

```
hostname: str = "server01"
ports: list[int] = [22, 443]
owner: str | None = None


def host_label(hostname: str, site: str | None = None) -> str:
  """Return a hostname label, optionally including its site."""
  if site is None:
    return hostname
  return f"{hostname} ({site})"
```

Docstrings

Purpose

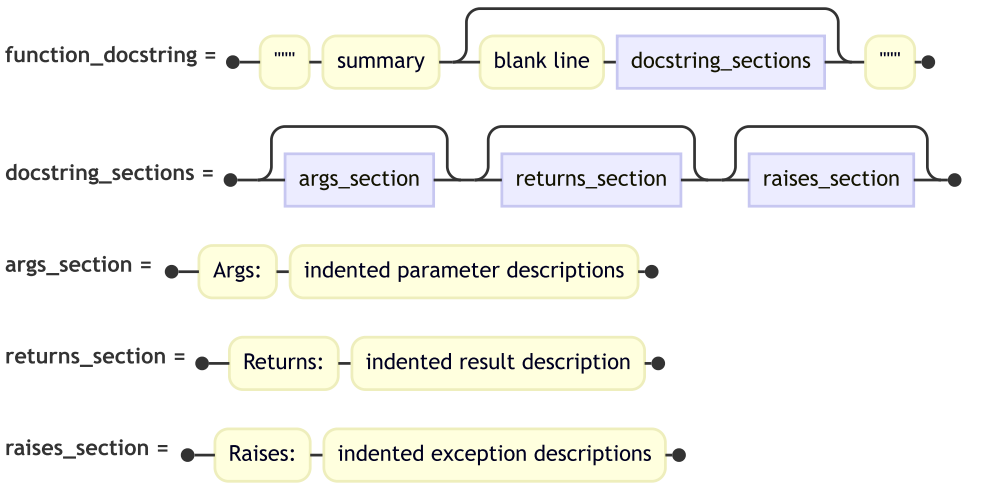
Use docstrings to explain a module, class, or function. A function's docstring describes its purpose, parameters, return value, and possible exceptions.

Notes

- A docstring is a string placed as the first statement in a module, class, or function. Use triple double quotes.
- Begin with a brief summary. A simple function may need only this line.
- For more detail, leave a blank line after the summary and use Google-style sections:
  - `Args:` describes parameters.
  - `Returns:` describes the result.
  - `Raises:` describes exceptions callers should expect.
- Include only the sections that apply; do not add doctests.
- Type annotations describe types; docstrings explain meaning and behavior.
- Use `help(function_name)` to view a function's documentation.

Syntax

Diagram



Examples

```python
def host_label(hostname: str, site: str | None = None) -> str:
  """Return a hostname label, optionally including its site.

  Args:
    hostname: Host name to label.
    site: Optional site name.

  Returns:
    The hostname, optionally followed by the site.
  """
  if site is None:
    return hostname  # "server01"
  return f"{hostname} ({site})"  # "server01 (Vancouver)"
```

Interactive CLI Input

Purpose

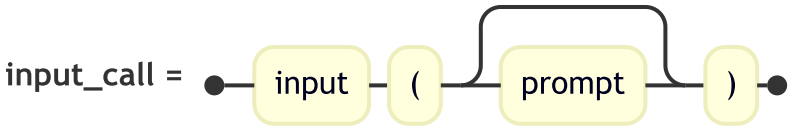
Get data interactively via the command line.

Notes

- `input()` always returns a `str`, even when the user types digits.
- Convert numeric text with `int()` or `float()`.
- Handle invalid text with `ValueError` when the program can recover.

Syntax

Diagram



Examples

```
# input reads the host and port as strings.
# Convert the port in try because invalid numeric text raises ValueError.
hostname = input("Host name: ")
port_text = input("Port number: ")

try:
  port = int(port_text)
except ValueError:
  print("Enter the port as a whole number.")
else:
  print(f"Checking {hostname} on port {port}")  # Example: Checking server01 on port 443
```

Variable Scope and Modifiers

Purpose

Understand where names belong and use `global` or `nonlocal` when a function must reassign a name outside its own local scope.

Notes

Variable Scopes

Names used without a prefix, such as `hostname`, are looked up in LEGB [Local, Enclosing, Global, Built-in] order:
local, enclosing function, global module, then built-ins.

Module Scope

Names defined at the top level are global to that module, not to every module in the program.

Function Scope

Parameters and names assigned in a function are local to that function call unless declared `global` or `nonlocal`.

Enclosing Function Scope

A nested function can read names from its enclosing functions; use `nonlocal` to reassign an existing name there.

Class and Object Attributes

Class Namespace

Assignments in a class body define class attributes, accessed as `ClassName.name`.

A method does not search the class namespace for names without a prefix; use `self.name` or `ClassName.name`.

Object (Instance) Attributes

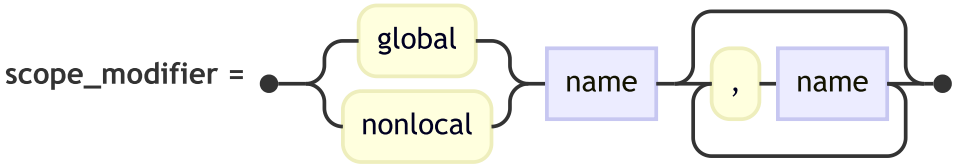
Assignments such as `self.name = value` store state on that instance.
They are attributes, and must be accessed with the object prefix and dot `.` operator.

Scope Modifiers

- `global name` directs assignments to the name in the current module.
- `nonlocal name` directs assignments to the nearest enclosing function scope that already binds the name; it cannot target a module or class attribute.
- Put declarations before using or assigning the name in that function.
- Reading an outer name does not require either declaration. Mutating an existing object, such as `items.append(value)`, is different from reassigning the name and does not require them either.
- `global` and `nonlocal` are not needed when assigning attributes, such as `self.name` or `ClassName.name`.
- Prefer parameters and return values over changing module-level state.

Syntax

Diagram



Examples

```
host_count = 2  # Global: module-level name.

def report_scope() -> None:
  host_count = 3  # Local: new name for this function call.
  print(host_count)  # 3


def add_host() -> None:
  global host_count
  host_count += 1  # Changes the module-level name.


def make_counter():
  count = 0  # Enclosing function scope.

  def next_count() -> int:
    nonlocal count
    count += 1  # Changes the enclosing function's name.
    return count

  return next_count


report_scope()
add_host()
counter = make_counter()
print(host_count)  # 3
print(counter(), counter())  # 1 2

class HostGroup:
  """Keep a shared category and an independent name for each group."""

  category = "network"

  def __init__(self, name: str) -> None:
    """Initialize this group's instance attribute."""
    self.name = name


first = HostGroup("Production")
second = HostGroup("Testing")
HostGroup.category = "infrastructure"  # Changes the class attribute.
first.name = "Live"                    # Changes only this instance.
print(first.category, second.category)  # infrastructure infrastructure
print(first.name, second.name)          # Live Testing
```

Code Layout

Organize code within scripts and across modules and packages so it is easy to find, reuse, and run.

Modules and Imports

Purpose

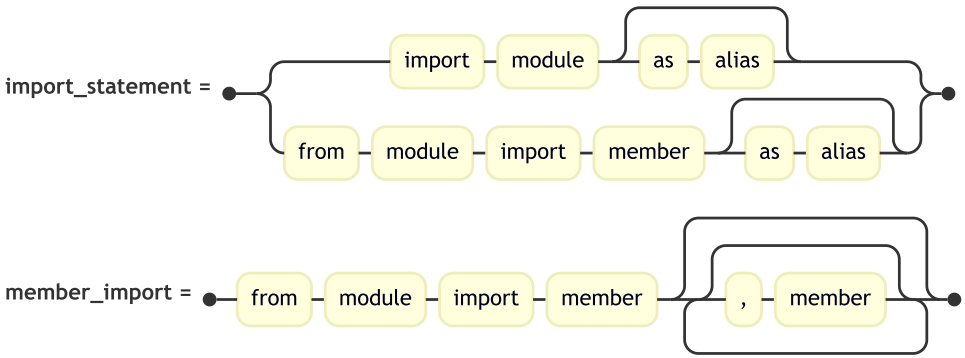
Use modules to organize reusable code and imports to access code from other modules.

Notes

- A module is commonly a `.py` file; Python also provides built-in modules.
- `import module` creates a name for the module in the importing module's namespace; access its functions and variables as `module.member`.
- `import module as alias` creates the alias; access members as `alias.member`.
- `from module import member` creates a direct name for that member, while `from module import member as alias` creates only the alias.
- An import inside a function creates a function-local name; a top-level import creates a module-level name.
- Avoid `from module import *` because it obscures where names came from.

Syntax

Diagram



Examples

```
# Each form binds a name in this module's namespace; comments show how to use it.
import math

# access members of module via `.` operator ie. math.sqrt(...), math.pi.
square_root = math.sqrt(25)

import math as maths
# Binds the alias maths; access members using alias maths.sqrt(...), maths.pi.
circle_area = maths.pi * 2 ** 2

from math import sqrt
# Binds sqrt directly; call sqrt(...) without a module prefix.
another_root = sqrt(36)

from math import pi as circle_pi
# Binds the alias circle_pi directly; use circle_pi without a module prefix.
circumference = 2 * circle_pi * 2
# square_root is 5.0; circle_area is approximately 12.57; another_root is 6.0.
# circumference is approximately 12.57.
```

Packages

Purpose

Use a package to group related modules under a shared name, making them easier to reuse and distribute together.

Notes

- A regular package directory contains an `__init__.py` file, which can be empty.
- Import a module from the package, then access its members using that module's name.
- An imported package is not the same as the installed distribution name: `uv add` installs a distribution, whose name may differ from its import name.

Syntax

```
from package import module
```

Examples

```
project/
  main.py
  host_tools/
    __init__.py
    checks.py
```

In the project's `main.py`:

```
from host_tools import checks

checks.function_name()  # Calls a function defined in host_tools/checks.py.
```

Recommended Script Structure

Purpose

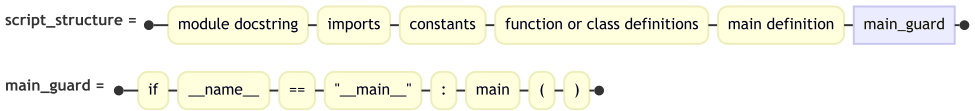
Organize a script so its reusable definitions can be imported without running its main task.

Notes

- Begin with a module docstring, followed by imports, constants, and function or class definitions.
- Put the script's main task in a function named `main`.
- When run directly, a module's `__name__` is `"__main__"`; when imported, it is the module's name.
- The `if __name__ == "__main__":` guard calls `main()` only when run directly. Other top-level statements still execute when the module is first imported.

Syntax

Diagram



Examples

```
"""Display the local host name."""

import socket

REPORT_TITLE = "Host report"


def main() -> None:
  """Run the host report."""
  print(f"{REPORT_TITLE}: {socket.gethostname()}")


if __name__ == "__main__":
  main()
```

uv Projects and Virtual Environments

Purpose

Use a project manager and virtual environment to keep each project's Python version and third-party packages isolated, recorded, and reproducible.

Notes

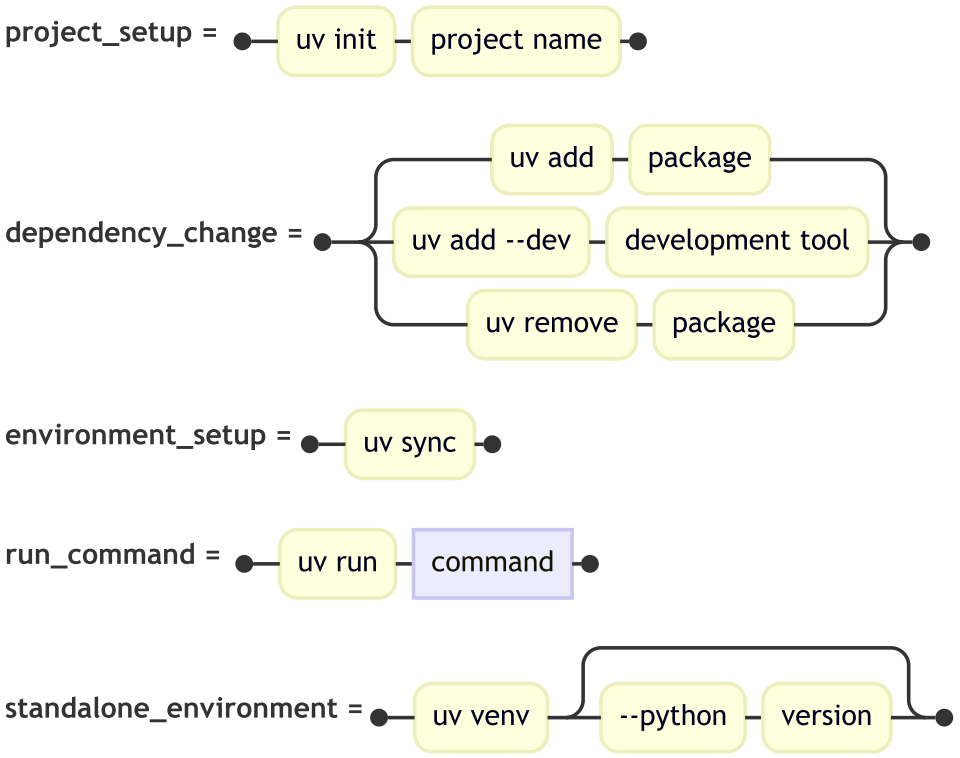
- `pyproject.toml` stores project metadata and direct dependency declarations; `uv.lock` records the resolved dependency versions.
- The project environment is normally stored in `.venv`. `uv run` uses it without requiring manual activation.
- Development dependencies, such as pytest, support development and testing; runtime dependencies, such as requests, are used by the application.
- After cloning or receiving a project, `uv sync` recreates its environment from the project files and lockfile.

Common Commands

- `uv init project_name` : creates a project with initial files and metadata.
- `uv add package` : adds a runtime dependency to the project.
- `uv add --dev package` : adds a development dependency.
- `uv remove package` : removes a runtime dependency.
- `uv sync` : synchronizes the project environment with its dependencies.
- `uv run command` : runs a command in the project environment.
- `uv venv --python 3.14` : creates a standalone virtual environment using Python 3.14, without creating project metadata.

Syntax

Diagram



Examples

```
# Run from the project folder: init creates files, add records dependencies,
# sync prepares the environment, and run executes commands in that environment.
uv init host_report
cd host_report
uv add requests
uv add --dev pytest
uv sync
uv run python main.py
uv run pytest  # Runs the test suite and reports the test results.


# For a standalone environment without project metadata, use uv venv.
uv venv --python 3.14
# Creates a standalone .venv using Python 3.14.
```

pytest Basics

Purpose

Use pytest tests to check expected program behavior automatically and repeatably as code changes.

Notes

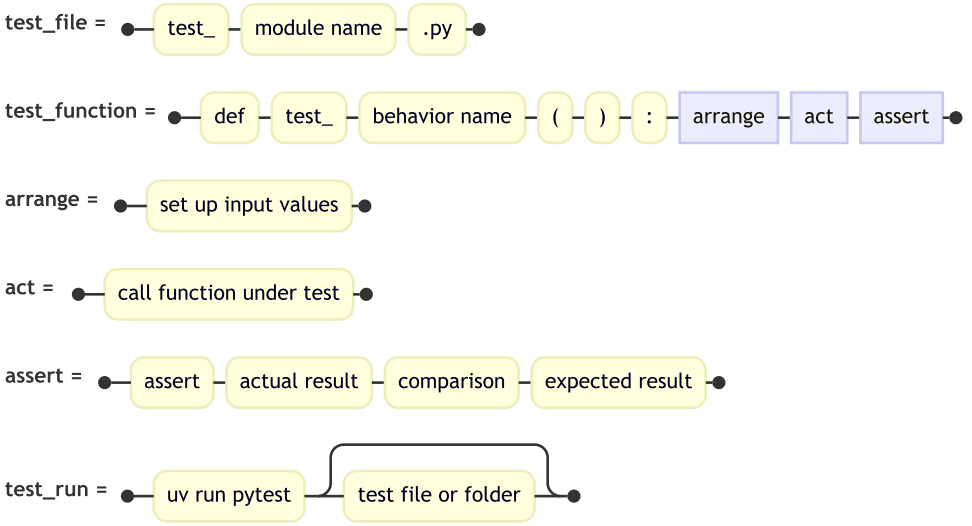
- Test file names start with `test_`
- test function names with `test_` .
- Organize each test as Arrange (set up inputs), Act (run the behavior), and Assert (check the result).
- Use `assert` to compare actual behavior with expected behavior.
- A false `assert` condition raises `AssertionError` and pytest reports the test as failed. An optional message, such as `assert result is True, "Expected port to be available"` explains the check.

Common Commands

- `uv run pytest` : runs the project's test suite.
- `uv run pytest tests/test_checks.py` : runs tests in one file.
- `uv run pytest tests` : runs tests in the specified folder.

Syntax

Diagram



Examples

```python
# test_checks.py
# The test_ names follow pytest discovery; the body uses Arrange, Act, Assert.
from checks import is_port_available # make sure to import the code to be tested


def test_port_not_in_open_ports_is_available():
  # Arrange: set up the test inputs.
  open_ports = [22, 443]

  # Act: run the code being tested.
  result = is_port_available(80, open_ports)

  # Assert: check the expected result.
  assert result is True  # Passes because port 80 is not already open - i.e. its available
```

Standard Library Modules

Python's standard library is installed with Python and needs no `uv add`. Import a module before using it; see Modules and Imports.

pathlib Module

Purpose

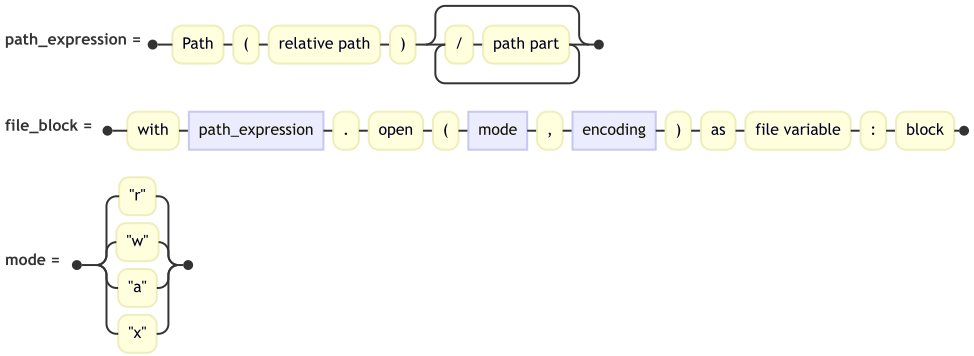
Use `pathlib.Path` to construct portable paths and file context managers to read or write data while ensuring the file is closed.

Notes

- File modes: `"r"` reads, `"w"` creates or replaces, `"a"` appends, and `"x"` creates only if absent.
- Specify UTF-8 for text files.
- Use `.read()` for a small whole file; iterate over a file for line-by-line processing.
- `.write()` does not add a newline automatically.
- Useful `Path` attributes and methods include `.exists()`, `.is_file()`, `.is_dir()`, `.name`, `.suffix`, and `.parent`.

Syntax

Diagram



Examples

```
# Demonstrates path_expression, directory_setup, and file_block.
from pathlib import Path

log_path = Path("logs") / "startup.log"
log_path.parent.mkdir(exist_ok=True)

with log_path.open("a", encoding="utf-8") as log_file:
  log_file.write("System check started\n")

with log_path.open("r", encoding="utf-8") as log_file:
  for line in log_file:
    print(line.strip())  # System check started
```

json Module

Purpose

Use JSON to store and exchange structured data, including nested Python lists and dictionaries.

Notes

- `json.load(file)` reads JSON from an open file and returns Python data.
- `json.loads(string)` reads JSON from a string and returns Python data.
- `json.dump(data, file)` writes Python data to an open file and returns `None`.
- `json.dumps(data)` converts Python data to a JSON string and returns that string.
- Recommended encoding is `encoding="utf-8"` for text files.
- `indent=2` makes written JSON easier to read without taking up too much space.

Syntax

Examples

```
# json_read loads file data into Python; json_write dumps Python data to a file.
import json

with open("tasks.json", "r", encoding="utf-8") as json_file:
  tasks = json.load(json_file)  # Python list or dictionary from the JSON file.

with open("tasks_out.json", "w", encoding="utf-8") as json_file:
  json.dump(tasks, json_file, indent=2)  # Writes formatted JSON; no return value.

json_text = json.dumps(tasks, indent=2)  # JSON text string.
tasks_copy = json.loads(json_text)       # Python list or dictionary.
```

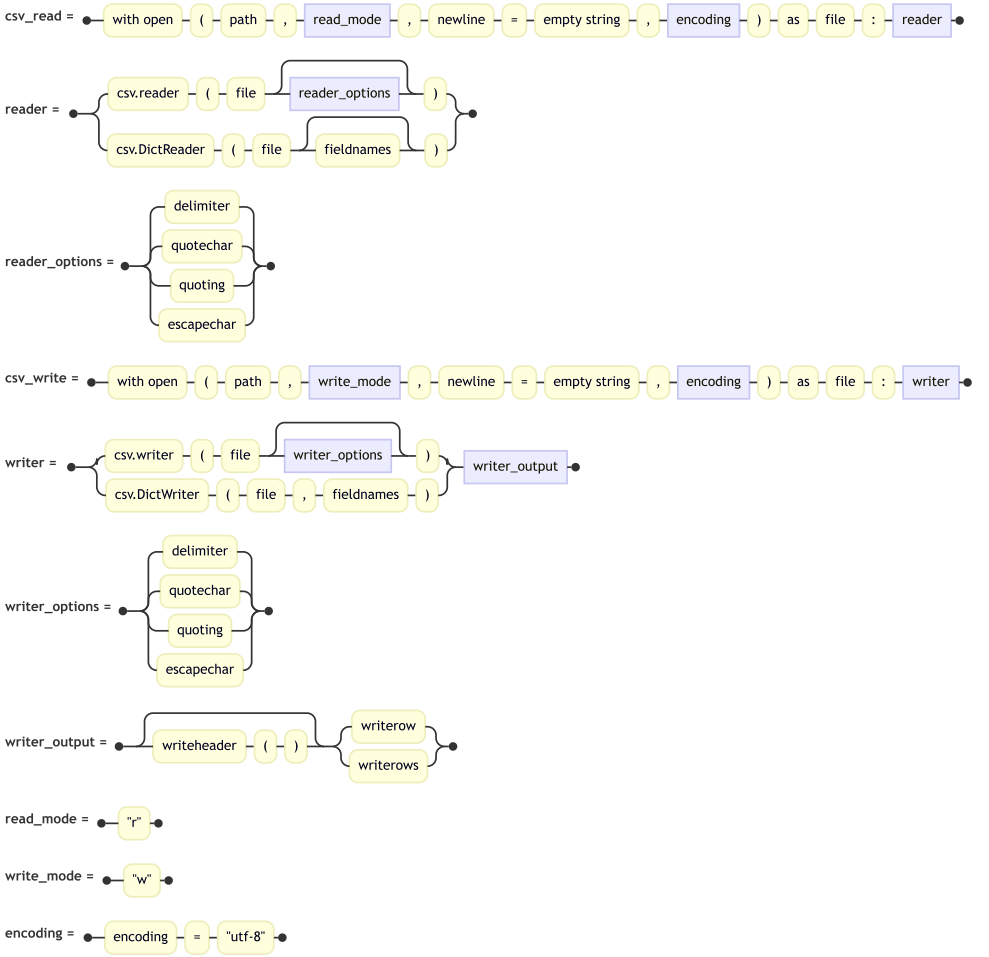
csv Module

Purpose

Use CSV for tabular data stored as rows and columns, especially when data needs to be opened in spreadsheet software.

Syntax

Diagram



Examples

```
# csv_read uses header names as keys; csv_write takes field names and writes a header.
import csv

with open("hosts.csv", "r", newline="", encoding="utf-8") as csv_file:
  reader = csv.DictReader(csv_file)
  for row in reader:
    print(row["hostname"], row["ip_address"])  # Prints each host and address.

fieldnames = ["hostname", "ip_address"]
with open("report.csv", "w", newline="", encoding="utf-8") as csv_file:
  writer = csv.DictWriter(csv_file, fieldnames=fieldnames)
  writer.writeheader()
  writer.writerow({"hostname": "server01", "ip_address": "192.168.1.10"})  # Writes one row.

# csv.reader returns each row as a list of strings.
with open("hosts.csv", "r", newline="", encoding="utf-8") as csv_file:
  reader = csv.reader(csv_file)
  for row in reader:
    print(row[0], row[1])  # Prints the hostname and IP address by position.
```

```
# DictReader can use supplied fieldnames when the file has no header row.
fieldnames = ["hostname", "ip_address"]
with open("hosts_without_header.csv", "r", newline="", encoding="utf-8") as csv_file:
  reader = csv.DictReader(csv_file, fieldnames=fieldnames)
  for row in reader:
    print(row["hostname"], row["ip_address"])  # Uses the supplied keys.

# Configure delimiters and quoting when fields contain separators.
with open("service_report.csv", "w", newline="", encoding="utf-8") as csv_file:
  writer = csv.writer(
    csv_file,
    delimiter=";",
    quotechar='"',
    quoting=csv.QUOTE_MINIMAL,
  )
  writer.writerow(["service", "owner", "status"])
  writer.writerow(["Web; API", "Operations", "running"])
  # Writes semicolon-separated fields and quotes the service field.
```

Notes

- CSV is plain text arranged as rows and fields; the delimiter separates fields and the first row commonly contains column names.
- Use `newline=""` when opening CSV files.
- `csv.reader` returns each row as a list; `csv.DictReader` returns each row as a dictionary usinq the header row as keys by default.
- Pass `fieldnames` to `DictReader` when the input has no header row.
- `csv.writer` writes list rows; `csv.DictWriter` writes dictionary rows and requires `fieldnames`.
- `DictWriter.writeheader()` writes the field-name row before data rows.
- `delimiter` selects the field separator; common alternatives include comma, tab, colon, and semicolon.
- `quotechar` surrounds fields containing delimiters; `quoting` controls when fields are quoted, such as `csv.QUOTE_MINIMAL`, `QUOTE_ALL`, `QUOTE_NONNUMERIC`, or `QUOTE_NONE`.
- `escapechar` escapes delimiters when quoting is not being used.
- Keep dictionary keys consistent with the writer's `fieldnames`.

sys Module

Purpose

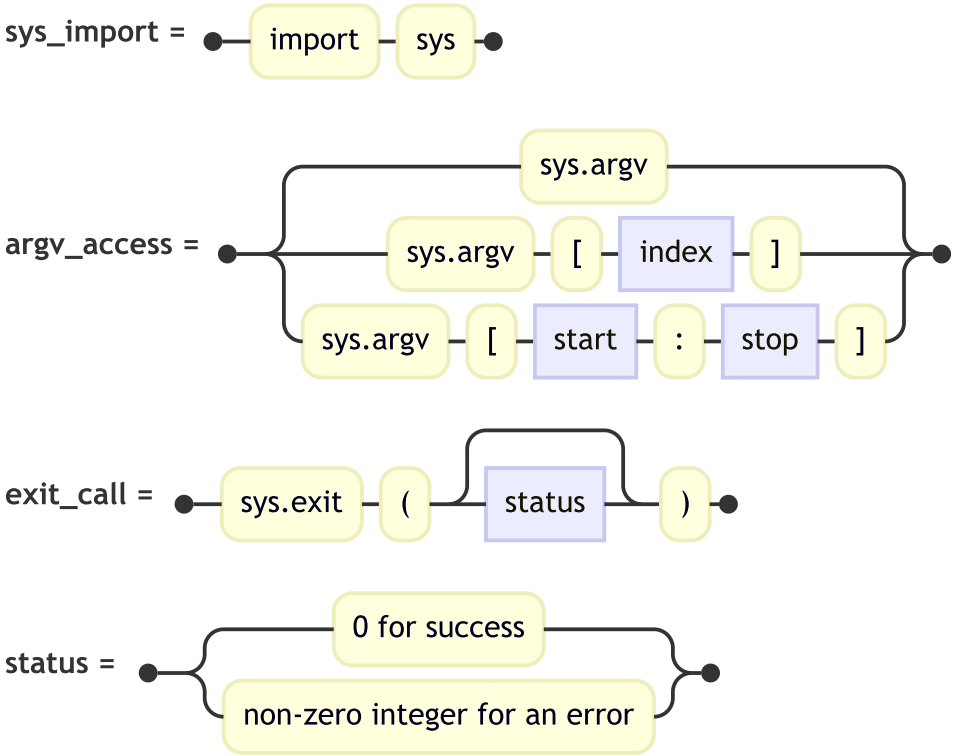
Use the `sys` module to access information about the running Python program and its interpreter, such as command-line arguments, and to control how the program ends.

Notes

- `sys` is a built-in module; use `import sys` before using it.
- Variables and functions used in this course:
  - `sys.argv`: a list of strings holding the command-line words supplied when the script was started.
  - `sys.exit(status)`: a function that stops the program.
- `sys.argv[0]` is the script name.
- `sys.argv[1:]` contains user-supplied arguments.
- Check the argument list length before indexing it.
- All elements of `argv` are strings, explicitly convert them when they represent numbers.
- `sys.exit(0)` signals success, a non-zero status such as `sys.exit(1)` signals an error.
- `sys.exit()` works by raising `SystemExit`, which ends the program unless it is caught.

Syntax

Diagram



Example

```
# Filename: show_host.py
# Command-line values are strings until explicitly converted.
import sys

arguments = sys.argv[1:]
if len(arguments) < 1:
  print("Usage: python show_host.py <hostname> [--verbose]")
  sys.exit(1)

hostname = arguments[0]
verbose = False

if len(arguments) > 1:
  flag = arguments[1]
  if flag == "--verbose":
    verbose = True
  else:
    print(f"Unknown option: {flag}")
    sys.exit(2)

print(f"Checking {hostname}")
if verbose:
  print("Verbose output enabled")  # Prints when --verbose was supplied.
```

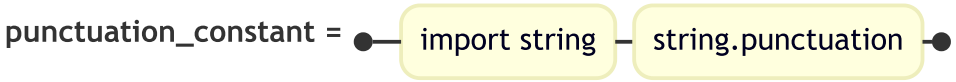
`string` Module

Purpose

Use the `string` module's punctuation constant when processing text and removing punctuation from input.

Syntax

Diagram



Examples

```
# Use string.punctuation to build excluded characters, then process input with a loop.
import string

excluded = set(string.punctuation + " ")
clean_text = ""
for character in "Server, ONLINE!":
  if character not in excluded:
    clean_text += character.lower()
# clean_text is "serveronline".
```

Notes

- `string.punctuation` contains the punctuation characters used by this demo.